



A general solver for electrochemical transport-reaction systems

Declare the system; the solver handles the math — demonstrated on ORR selectivity

Jake Weinstein

■ One framework, many systems

Migration–diffusion–reaction with a structured double layer — the same mathematics across electrochemistry.

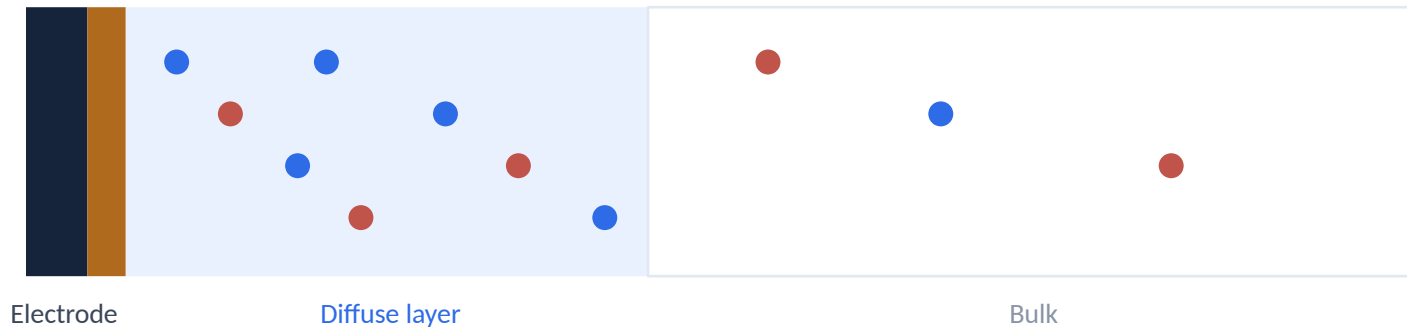
ORR

CO₂RR

Electrodeposition

Corrosion

Stern



We'll demonstrate the workflow on ORR.

■ Two halves of the problem

Model specification

the electrochemistry

- Species (dynamic & analytic)
- Reactions & stoichiometry
- Extra physics terms
- Boundary conditions



Numerical machinery

the solver's job

- Formulation & log-space
- Meshing (1D / 2D)
- Continuation & recovery
- Robust convergence

■ What you specify

PNP in the diffuse layer — ions diffuse and electromigrate in the field.

$$\frac{\partial c_i}{\partial t} = \nabla \cdot \left(D_i \left[\nabla c_i + \frac{z_i F}{RT} c_i \nabla \varphi \right] \right) \quad - \nabla \cdot (\varepsilon \nabla \varphi) = \sum_i z_i F c_i$$

Butler-Volmer flux — rate scales with $\exp(\text{overpotential})$, tied to the applied voltage.

$$R_r = k_{0,r} \left[c_O e^{-\alpha_r F \eta_r / RT} - c_R e^{(1-\alpha_r) F \eta_r / RT} \right], \quad \eta_r = \varphi_m - \varphi_s - E_{,r}^\circ$$

Steric correction — removes non-physical ion concentrations.

$$\mu_i^{\text{steric}} = -\ln \theta, \quad \theta = 1 - \sum_i a_i c_i$$

Stern layer — enters as a Robin boundary condition.

$$\varepsilon \partial_n \varphi = C_S (V_{\text{app}} - \varphi_{\text{OHP}}), \quad C_S = 0.20 \text{ F/m}^2$$

What you specify

Bulk Dirichlet

Butler-Volmer flux

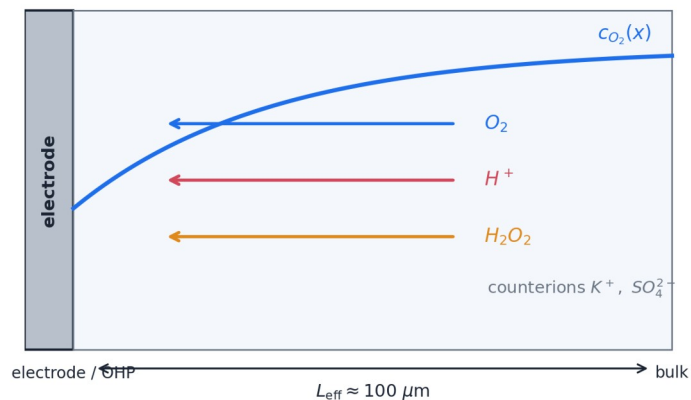
Stern Robin

No-flux

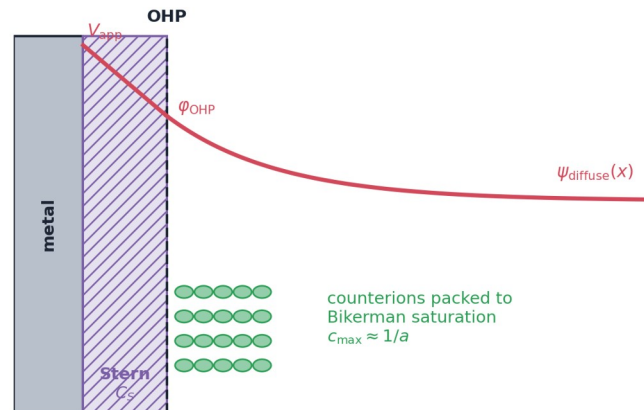
Analytic Bikerman

Model domain & the double layer — where the numerical difficulty lives

Quasi-1D diffusion layer



Compact + diffuse layer (the zoom)



Mix and match per species, per boundary.

■ What you specify



$E^\circ = 0.695 \text{ V} \cdot 2\text{e}^-$ — the target



$E^\circ = 1.23 \text{ V} \cdot 4\text{e}^-$ — the loss

Declare stoichiometry · n_e · E° · α · k_o → log-rate Butler-Volmer (parallel or sequential)

Edit a coefficient or append a reaction — the flux re-assembles.

■ Why it's numerically hard

Concentrations span ~10 orders of magnitude across the domain.

The potential varies sharply right at the Stern layer.

The exponential Butler-Volmer term drives steep buildup in c and φ at the boundary.

Diffusion and electromigration act on very different timescales — the system is stiff.

$$c_i \approx c_i^{\text{bulk}} \exp\left(-z_i F \varphi / RT\right)$$

$$\lambda_D = \sqrt{\frac{\varepsilon RT}{2F^2 I}} \ll L_{\text{eff}}, \quad I = \frac{1}{2} \sum_i z_i^2 c_i$$

How we solve it

1 Solve $\mu = \ln c + z \cdot \varphi$ — mirrors the no-flux analytic solution.

$$\mu_H = u_H + \frac{z_H F}{RT} \varphi, \quad c_H = \exp\left(\mu_H - \frac{z_H F}{RT} \varphi\right), \quad \mathbf{J}_H = -D_H c_H \nabla \mu_H$$

2 Log-rate Butler-Volmer — build the rate in log-space, exponentiate once (no overflow).

$$\log r_r^{(\text{cat})} = \ln k_{0,r} + u_O - \alpha_r \frac{F \eta_r}{RT}, \quad \log r_r^{(\text{anod})} = \ln k_{0,r} + u_R + (1 - \alpha_r) \frac{F \eta_r}{RT}, \quad R_r = r_r^{(\text{cat})} - r_r^{(\text{anod})}$$

3 Analytic closure for non-reactive ions.

$$c_b(x) = c_b^{\text{bulk}} \frac{(1 - \sum_{j \in \text{dyn}} a_j c_j) e^{-z_b F \varphi / RT}}{\theta_b + a_b c_b^{\text{bulk}} e^{-z_b F \varphi / RT}}, \quad \theta_b = 1 - \sum_j a_j c_j^{\text{bulk}} - a_b c_b^{\text{bulk}}$$

4 Continuation — never solve the hard problem cold (next slide).

Continuation: deform easy → hard

Reaction rate k_0 : ramp from $\sim 10^{-12} \times$ physical up to physical, warm-starting each step.

Stern capacitance: ramp up (stiffer as C_S grows).

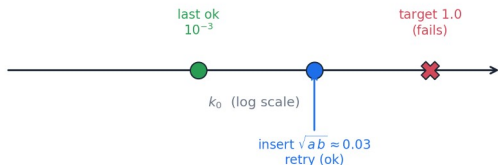
Voltage walk: converge near the equilibrium voltage, then step out across the window.

Recursive bisection: insert intermediate steps when one fails — recover, don't restart.

How a failed continuation step recovers

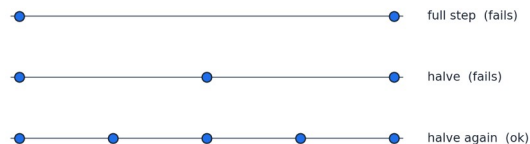
1 · failed rung → geometric-midpoint retry

step too big in log-space?
halve it (geometric midpoint), then go on



2 · cliff: recursively halve the voltage step

warm-walk: 8 substeps $\times$ depth-5 (up to 32x refine)

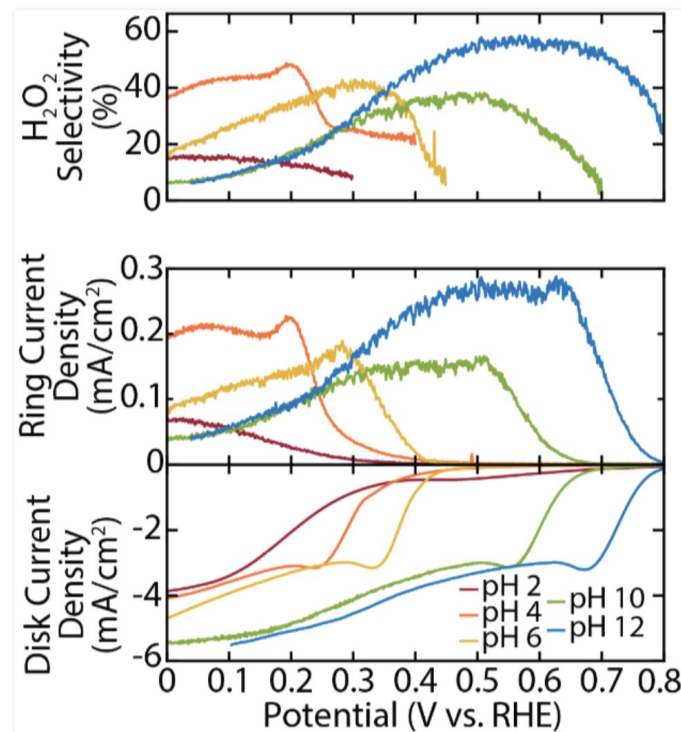


each sub-step warm-started → walks across the near-discontinuity

■ Proof of concept: ORR selectivity

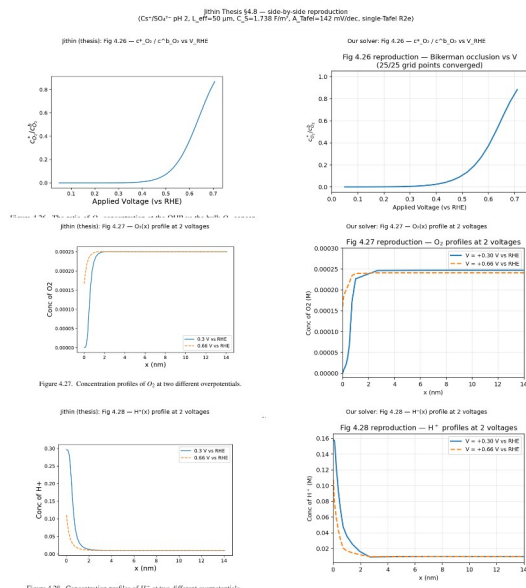
The system, declared from the menu:

- Parallel $2e^-$ (0.695 V) + $4e^-$ (1.23 V)
- K^+ / SO_4^{2-} electrolyte
- Finite-size ions (Bikerman)
- Stern + Butler-Volmer + bulk BCs

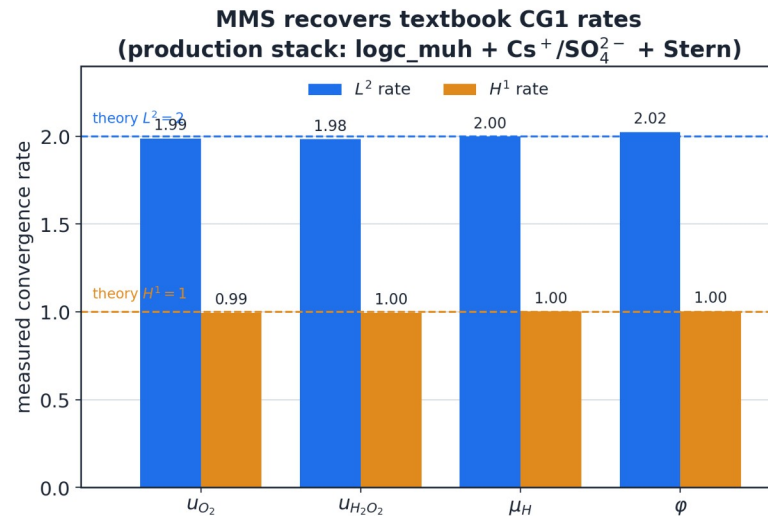


Target: Ruggiero / Seitz-Mangan, pH 2-12

What's proven — and what's left



Reproduces the prior model (~5%)



MMS → textbook convergence

The remaining gap to experiment is model selection — not the solver.

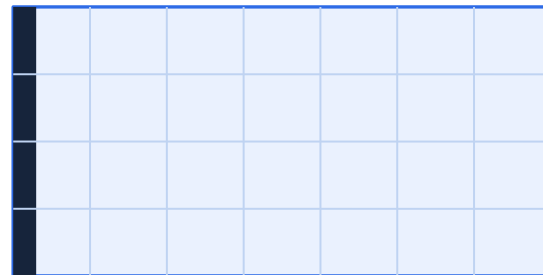
■ 1D and 2D – both today

1D



fast fitting & parameter sweeps

2D



real cell geometry & radial transport

Same governing equations — running in 2D today.

■ Where it's headed: a Python package

```
import pnpbv as pb

O2 = pb.Species("O2", z=0, D=2.1e-9, c_bulk=1.2e-3)
H  = pb.Species("H+", z=+1, D=9.3e-9, c_bulk=1e-4)
K  = pb.Species("K+", z=+1, c_bulk=0.2,
                analytic=True)          # Boltzmann

r2e = pb.BVReaction("ORR_2e",
                    reactants={O2:1, H:2}, products={H2O2:1},
                    n_e=2, E_eq=0.695)

prob = pb.Problem(
    species=[O2, H2O2, H, K, S04],
    reactions=[r2e, r4e],
    electrode=pb.SternElectrode(C_stern=0.20))

sol = pb.Solver(prob).solve_grid(v_rhe=(-0.4, 0.8))
```

Declare the spec.

It handles meshing,
formulation, solving,
and recovery.

Planned: hyperparameter
autotuning.

Design sketch — not yet built

■ Extend it: add your own term

```

from pnpbv.formulations import pnp_steric
from ufl import grad, inner

# default PNP + migration + Bikerman steric:
res = pnp_steric(prob)
c, w = res.fields, res.tests    # conc + tests
dx = res.dx                     # 2D bulk measure

# add terms the menu doesn't cover (F = 0):
# peroxide decomposition: H2O2 -> 1/2 O2 + H2O
res.add( -k_d * c["H2O2"] * w["H2O2"] * dx )
res.add( +0.5*k_d * c["H2O2"] * w["O2"] * dx )
# forced convection (RDE):
res.add( inner(u, grad(c["O2"])) * w["O2"] * dx )

# same continuation + failed-step recovery:
sol = pb.Solver.from_weak_form(res).solve_grid(...)

```

Physics not on the menu?

Import the default residual;
append terms as plain UFL.

One line per term.

The same solver, continuation,
and recovery ride along.

Design sketch — not yet built



Roadmap

1

Close the experiment gap by model selection

candidate mechanisms: cation hydrolysis / field-dependent pKa, peroxide reduction, water self-ionization

2

Broaden the menu & add autotuning

more terms, BCs, and reactions; let the solver pick its own settings

3

Lock the API

a reusable predictive simulator for systems in this family